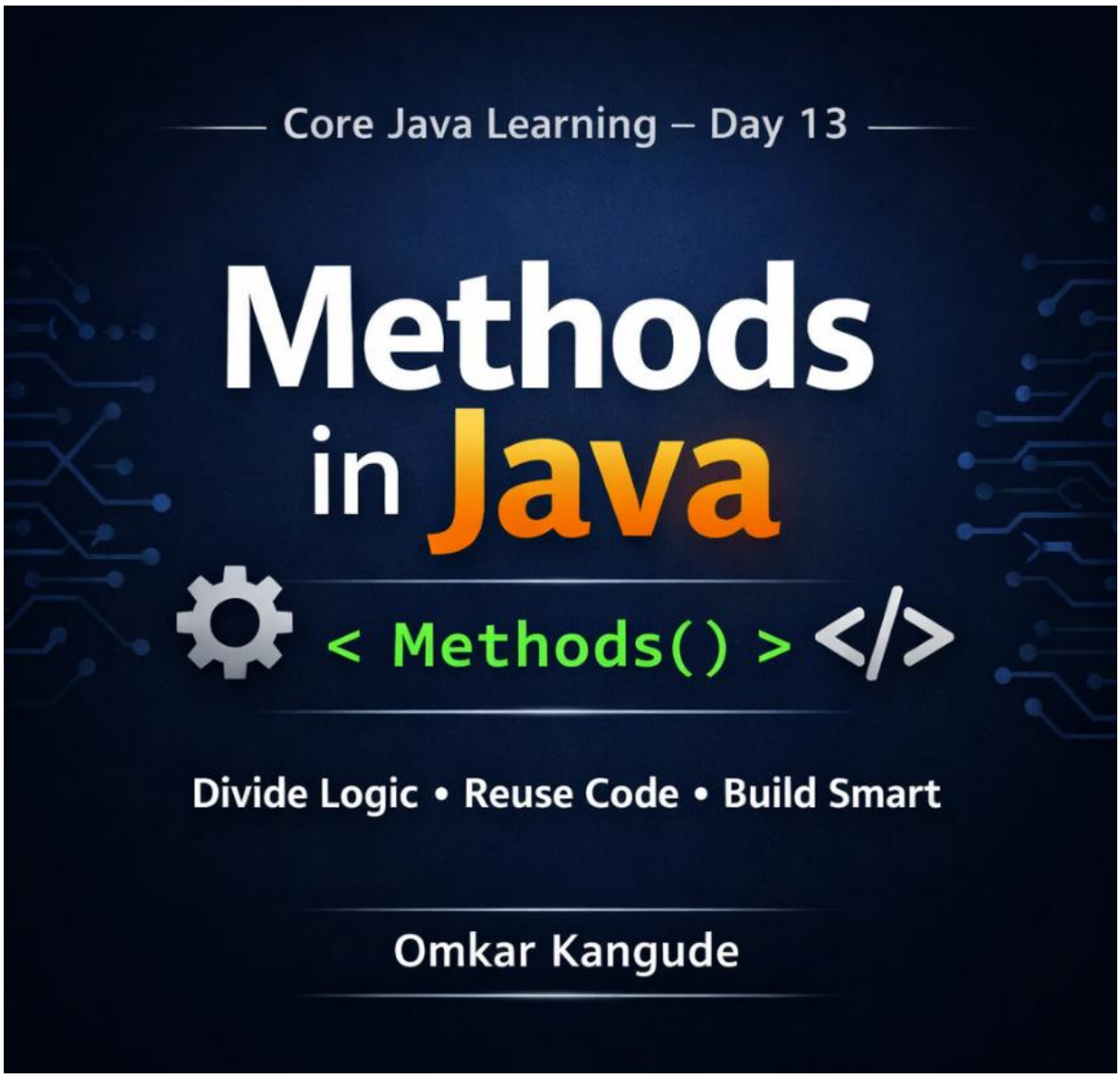
Methods In JAVA



1. What is a Method in Java

A method in Java is a block of code that is designed to perform a specific task. It is one of the most important building blocks in Java programming because it allows developers to break down complex problems into smaller, manageable parts.

Instead of writing the same code again and again, we can write it once inside a method and reuse it multiple times. This not only saves time but also makes the program more organized and readable.

From a deeper perspective, a method represents the behavior of an object. For example, if we consider a real-world object like a car, it has behaviors such as start, stop, accelerate, and brake. In Java, these behaviors are implemented using methods.

Methods also help in modular programming. Modular programming means dividing a program into smaller parts so that each part can be developed, tested, and maintained independently.

Example:

```
void startCar() {
   System.out.println("Car started");
}
```

2. Why Methods are Important in Java :-

Methods are a very important part of Java programming because they help in writing clean, organized, and efficient code. Without methods, a program would become very long, confusing, and difficult to manage. Methods allow us to divide a large program into smaller parts, where each part performs a specific task. This approach is known as modular programming.

One of the biggest advantages of methods is **code reusability**. Instead of writing the same logic multiple times, we can write it once inside a method and use it whenever needed. This reduces code duplication and saves time. For example, if we need to add two numbers in different parts of the program, we can create a single method for addition and reuse it.

Methods also improve **code readability**. When we use meaningful method names, it becomes easier to understand what the program is doing. For example, a method named calculateSalary() clearly indicates its purpose. This makes the code more readable and professional.

Another important benefit is **easy maintenance**. If we need to update any logic, we only have to change it in one place (inside the method), instead of changing it everywhere in the program. This reduces errors and makes updates easier.

Methods also help in **reducing complexity**. Large programs can be broken into smaller methods, making them easier to understand and manage. For example, in an e-commerce application, tasks like login, search, and payment can be written as separate methods.

Additionally, methods make **debugging easier**. If an error occurs, we can directly check the specific method instead of searching through the entire code. This saves time and effort.

Methods also support **teamwork** in real-world projects. Different developers can work on different methods, which speeds up development and improves collaboration.

Example

```
public class Example {

  static int add(int a, int b) {
    return a + b;
  }

  public static void main(String[] args) {
    int result1 = add(5, 10);
    int result2 = add(20, 30);

    System.out.println(result1);
    System.out.println(result2);
  }
}
```

3. Structure of a Method

A method in Java is made up of several important components that define how it works. Understanding the structure of a method is essential because it helps us write correct and efficient programs. Every method follows a specific format, which includes the return type, method name, parameters, and method body.

The first part of a method is the **return type**. It specifies what kind of value the method will return after execution. For example, a method can return an integer (int), a decimal number (double), a text value (String), or nothing (void). If a method does not return any value, we use the void keyword.

The second part is the **method name**. This is the name used to call the method. It should always be meaningful and follow Java naming conventions. A good method name clearly describes what the method does. For example, names like calculateTotal() or printDetails() are easy to understand.

The third part is **parameters**. Parameters are inputs that are passed to the method when it is called. They allow the method to work with different values. A method can have zero, one, or multiple parameters depending on the requirement.

The fourth part is the **method body**. This is the block of code inside curly braces { } where the actual logic is written. It contains statements that perform the required task.

The final part is the **return statement**. It is used to send the result back to the calling method. The return type and return value must match, otherwise it will cause an error.

Understanding this structure helps developers write clean and well-organized methods.

Example

```
public class Example {

 int multiply(int a, int b) {   // return type + method name + parameters
      int result = a * b;       // method body
      return result;          // return statement
   }

  public static void main(String[] args) {
      Example obj = new Example();
      int answer = obj.multiply(4, 5);
      System.out.println(answer);
   }
}
```

4. Method Execution Flow :-

When a method is called in Java, it follows a specific execution process managed by the JVM (Java Virtual Machine). Understanding this flow is very important because it helps us know how a program actually runs internally.

The execution starts from the main() method, which is the entry point of every Java program. When the program begins, the JVM starts executing the code inside the main() method line by line.

Whenever a method is called from the main() method (or any other method), the JVM creates a **stack frame** in memory. This stack frame is created inside a special memory area called the **stack memory**. The stack frame stores all the necessary information required to execute that method, such as parameters, local variables, and return value.

After the stack frame is created, the control moves from the calling method to the called method. The parameters passed to the method are copied into the stack frame, and then the method starts executing its code step by step.

If the method contains a return statement, it sends the result back to the calling method. Once the execution of the method is completed, the stack frame is removed from memory, and the control goes back to the previous method.

This process continues for every method call in the program. If multiple methods are called one after another, they are stored in the stack in a **Last In First Out (LIFO)** order. This means the last method called will finish execution first.

👉 **Example**

```
public class Example {

 static int add(int a, int b) {
      int result = a + b;          // execution happens here
      return result;               // value returned
   }
   public static void main(String[] args){


      System.out.println("Start");
    int sum = add(5, 3);                // method call → stack frame created
   System.out.println(sum);
      System.out.println("End");
   }
}
```

5. Types of Methods :-

In Java, methods can be classified into different types based on how they are defined and how they work. Understanding these types is important because it helps us choose the right method according to the requirement of the program.

The first type is predefined methods. These methods are already available in Java libraries and can be used directly without writing their implementation. Examples include System.out.println() and Math.sqrt(). These methods save time and make development faster because we do not need to write common functionalities from scratch.

The second type is user-defined methods. These methods are created by the programmer to perform specific tasks. Whenever the built-in methods are not enough, developers create their own methods based on the problem requirements.

Methods can also be classified based on parameters and return types. The first category is methods with no parameters and no return value. These methods simply perform an action and do not take any input or return any result.

The second category is methods with parameters but no return value. These methods take input values but do not return anything. They are mainly used for displaying or processing data.

The third category is methods with no parameters but return value. These methods do not take any input but return a result after performing some operation.

The fourth category is methods with parameters and return value, which are the most commonly used. These methods take input, process it, and return the result.

Understanding these types helps in writing flexible and efficient programs. It also improves code readability and makes the program more structured.

Example

```
public class Example {

    // No parameter, no return
    void show() {
        System.out.println("Hello Java");
    }

    // Parameter, return value
    int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        Example obj = new Example();

        obj.show();                    // calling first method
        int result = obj.add(10, 20);  // calling second method

        System.out.println(result);
    }
}
```

6. Method Overloading

Method overloading is a feature in Java that allows us to create multiple methods with the same name but different parameters. It is one of the important concepts of Java because it improves code readability and flexibility.

In method overloading, the method name remains the same, but the number of parameters, type of parameters, or order of parameters must be different. The compiler decides which method to call based on the arguments passed during method invocation. This process happens at compile time, so method overloading is also known as **compile-time polymorphism**.

The main purpose of method overloading is to perform similar types of operations using the same method name. For example, adding two numbers, three numbers, or decimal numbers can all be done using the same method name add(). This makes the program easier to understand and avoids confusion.

There are some important rules for method overloading. First, the method name must be the same. Second, the parameters must be different in terms of number, type, or order. Third, changing only the return type is not enough for overloading; the parameters must change.

Method overloading helps in writing cleaner and more organized code. Instead of creating multiple method names for similar operations, we can use a single name with different parameter combinations.

👉 **Example**

```
public class Example {

  int add(int a, int b) {
    return a + b;
  }

  int add(int a, int b, int c) {
    return a + b + c;
  }

  double add(double a, double b) {
    return a + b;
```

```
    }

    public static void main(String[] args) {
        Example obj = new Example();

        System.out.println(obj.add(5, 10));       // calls first method
        System.out.println(obj.add(5, 10, 15));    // calls second method
        System.out.println(obj.add(2.5, 3.5));     // calls third method
    }
}
```

7. Static vs Instance Methods

In Java, methods are mainly divided into two types: static methods and instance methods. Understanding the difference between these two is very important because it helps us decide how to design and use methods properly in a program.

A static method belongs to the class rather than the object. This means we do not need to create an object to call a static method. Static methods are declared using the static keyword. They are commonly used for operations that are not dependent on object data. For example, utility methods like mathematical calculations are usually static.

Static methods can be called using the class name. Since they belong to the class, they are shared among all objects of that class. However, static methods cannot directly access instance variables or instance methods because those belong to specific objects.

On the other hand, an instance method belongs to an object of a class. To call an instance method, we must first create an object of that class. Instance methods are used when we need to work with object-specific data.

Instance methods can access both instance variables and static variables. They can also call other instance methods and static methods. This makes them more flexible when working with object-oriented programming concepts.

In simple terms, static methods are used for common functionality, while instance methods are used for object-specific behavior.

Example

```
public class Example {

  static void staticMethod() {
    System.out.println("This is a static method");
  }

  void instanceMethod() {
    System.out.println("This is an instance method");
  }

  public static void main(String[] args) {

    // Calling static method (no object needed)
    Example.staticMethod();
```

```
        // Calling instance method (object required)
        Example obj = new Example();
        obj.instanceMethod();
    }
}
```

8. Pass by Value

In Java, all method arguments are passed using pass by value. This is a very important concept and is frequently asked in interviews. Many beginners get confused and think Java uses pass by reference, but that is not true. Java always passes a copy of the value, not the original variable.

When a variable is passed to a method, Java creates a copy of that variable and sends it to the method. Any changes made inside the method affect only the copy, not the original variable. This is why the original value remains unchanged after the method execution.

For primitive data types like int, double, char, etc., the actual value is copied. For example, if we pass an integer to a method and modify it inside the method, the original value outside the method will not change.

For objects, Java still uses pass by value, but the value being copied is the reference (address) of the object. This means both the original reference and the copied reference point to the same object in memory. So, if we modify the object's data inside the method, the changes will reflect outside the method as well. However, if we assign a new object inside the method, it will not affect the original reference.

This concept is important because it helps avoid unexpected changes in data and makes programs more predictable.

Example

```
public class Example {
    void changeValue(int x) {
      x = 100;   // change only in copy
   }
  public static void main(String[] args) {

      int a = 10;

    Example obj = new Example();
      obj.changeValue(a);

      System.out.println(a); // Output: 10 (unchanged)
   }
}
```

9. Recursion

Recursion is a programming technique in which a method calls itself to solve a problem. It is mainly used when a problem can be broken down into smaller subproblems of the same type. Instead of using loops, recursion allows us to solve problems in a more elegant and logical way.

In recursion, there are two very important parts: the base case and the recursive call. The base case is the condition where the recursion stops. Without a base case, the method will keep calling itself again and again, which can lead to a runtime error called Stack Overflow Error. The recursive call is the part where the method calls itself with a smaller or simpler input.

When a recursive method is executed, each method call is stored in the stack memory as a separate stack frame. As the method keeps calling itself, new stack frames are created. Once the base condition is reached, the method starts returning values, and the stack frames are removed one by one.

Recursion is very useful for solving problems like factorial calculation, Fibonacci series, tree traversal, and searching algorithms. However, it should be used carefully because too many recursive calls can consume a lot of memory.

In simple terms, recursion means solving a big problem by breaking it into smaller parts and solving each part using the same method.

👉 Example

```
public class Example {

  int factorial(int n) {
    if (n == 1) {
      return 1;            // base case
    }
    return n * factorial(n - 1);  // recursive call
  }

  public static void main(String[] args) {
    Example obj = new Example();

    int result = obj.factorial(5);
    System.out.println(result);   // Output: 120
  }
}
```

10. Real World Examples

Methods are not just theoretical concepts; they are used extensively in real-world applications. In every software system, methods help in organizing code by dividing complex tasks into smaller and manageable parts. This makes the system easier to develop, understand, and maintain.

In real-world applications like banking systems, e-commerce platforms, mobile apps, and websites, every action is performed using methods. For example, in a banking system, operations such as checking balance, withdrawing money, and depositing money are implemented using different methods. Each method performs a specific task, making the system structured and easy to manage.

Similarly, in an e-commerce application like Amazon or Flipkart, the entire process is divided into methods such as login, search product, add to cart, and payment. Instead of writing one large block of code, developers create separate methods for each functionality. This improves readability and allows developers to update or fix one part of the system without affecting others.

Methods also play an important role in mobile applications. For example, in a food delivery app, actions like selecting food, placing an order, and tracking delivery are handled by different methods. This modular approach makes the application more efficient and scalable.

In large companies, different developers work on different methods of the same project. This helps in faster development and better collaboration. Methods also make testing easier because each method can be tested individually.

In simple terms, methods are used in real-world systems to organize code, improve performance, and handle complex operations efficiently.

Example

```
public class ATM {

  void checkBalance(double balance) {
    System.out.println("Balance: " + balance);
  }

  void withdraw(double amount) {
    System.out.println("Withdrawn: " + amount);
  }

  public static void main(String[] args) {
```

```
        ATM obj = new ATM();

        obj.checkBalance(5000);
        obj.withdraw(1000);
    }
}
```